Run 4: Clique Ideas and Hub Links

October 27, 2026

What we tested

Our model is a relational GNN: it keeps separate weights for each type of edge. There are two edge types. The first is the dataset's own graph. The second ties the nodes of one subgraph together, in one of two ways (Figure 1):

- **Clique**: an edge between every two nodes of the same subgraph.
- **Extra node** (hub): one new node per subgraph, linked to all its members. The hub's vector is the subgraph's vector.

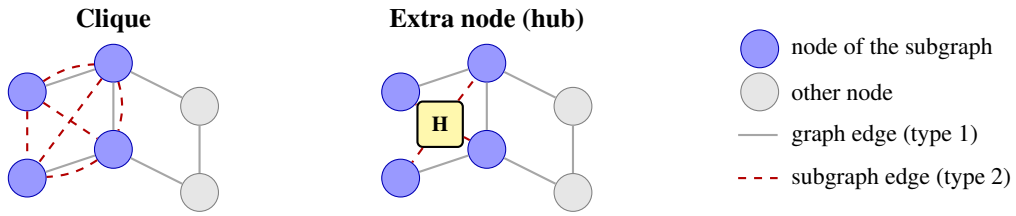


Figure 1: The two kinds of subgraph edge, on one subgraph of four nodes.

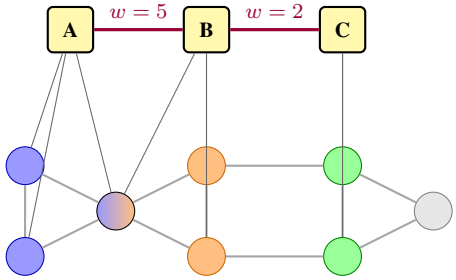
The advisor's concern (October 23) was that both kinds pull all nodes of a subgraph to one shared vector, which is close to plain mean pooling, and that this gets worse for big subgraphs. This run tests the advisor's suggestions and one idea from SubGNN:

No clique. Graph edges only. Shows what the graph alone gives.

Clique, symmetric. Today each node takes the plain mean of its clique neighbors. Here the message from node j to node i is divided by $\sqrt{d_i d_j}$ instead, where d is the number of clique neighbors (as in the original GCN).

Clique, symmetric $+ 1/N_C$**.** As above, and each clique edge of subgraph C is also multiplied by $1/N_C$, where N_C is the number of nodes in C. (With a plain mean the $1/N_C$ would cancel out, so it is only tried on top of the symmetric version.)

Hub graph. The extra-node model, plus a third edge type that links hubs of related subgraphs, so a subgraph also hears from other subgraphs (Figure 2).



node shared by A and B (half blue, half orange)

A–B: 1 shared node, plus 4 edges from an A node to a B node $\Rightarrow w = 5$

B–C: 0 shared nodes, 2 edges between $\Rightarrow w = 2$

A–C: nothing $\Rightarrow$ no link

Figure 2: Hub links (purple). The weight w of a link is the number of nodes the two subgraphs share plus the number of graph edges between them. Linking every related pair would make the hubs almost a full clique on most datasets, so each hub keeps only its 10 strongest links.

Every case starts from the best settings found for the plain clique (run 2), changes only what the case is about, searches a few learning rates, and then trains 10 seeds. The plain clique was also trained again with 10 new seeds. em_user was left out, as before.

Results

	clique models				extra-node models		published		
dataset	plain	none	symmetric	sym. $+1/N_C$	extra node	hub graph	SubGNN	GLASS	GPEN
ppi_bp	59.2 ±1.1	59.9 ±2.1	**60.0** ±1.9	59.4 ±2.9	**60.6** ±1.9	60.3 ±1.8	59.9 ±2.5	61.9 ±2.2	**64.4** ±0.9
hpo_metab	54.7 ±3.4	53.7 ±2.8	**55.3** ±2.4	53.7 ±3.3	52.4 ±4.3	**52.6** ±2.3	53.7 ±2.5	61.4 ±1.6	**63.9** ±0.9
hpo_neuro	62.4 ±0.9	62.7 ±1.0	**63.2** ±1.1	63.0 ±1.6	**60.9** ±1.2	60.3 ±0.9	64.4 ±1.9	68.5 ±1.6	**69.1** ±0.6
density	**78.8** ±6.5	70.8 ±3.1	75.2 ±4.7	72.8 ±3.9	67.2 ±5.0	**69.2** ±5.4	91.9 ±1.9	93.0 ±2.8	**95.6** ±1.6
cut_ratio	**75.6** ±4.9	73.2 ±3.6	73.2 ±4.0	**75.6** ±3.3	79.2 ±3.0	**81.6** ±2.7	62.9 ±4.1	93.5 ±1.9	**93.6** ±1.6
coreness	84.4 ±5.2	82.6 ±3.4	84.4 ±5.2	**84.8** ±4.9	76.5 ±3.5	**78.7** ±3.1	65.9 ±9.8	84.0 ±2.8	**87.6** ±1.9
component	**99.2** ±1.6	86.0 ±12.2	84.8 ±21.5	83.2 ±23.2	67.2 ±8.5	**92.0** ±3.6	95.8 ±10.1	**100.0** ±0.0	**100.0** ±0.0

Table 1: Test micro-F1 ($\times 100$), mean $\pm$ std over 10 seeds. "Plain" is today's clique with a plain mean; "none" has graph edges only. The extra-node column is from run 3, the rest are from this run. The published columns are copied from Tables 1 and 3 of the GPEN paper (10 runs each). Bold: best in each row within each group (clique, extra node, published); most gaps inside a group are smaller than the std.

Error against subgraph size

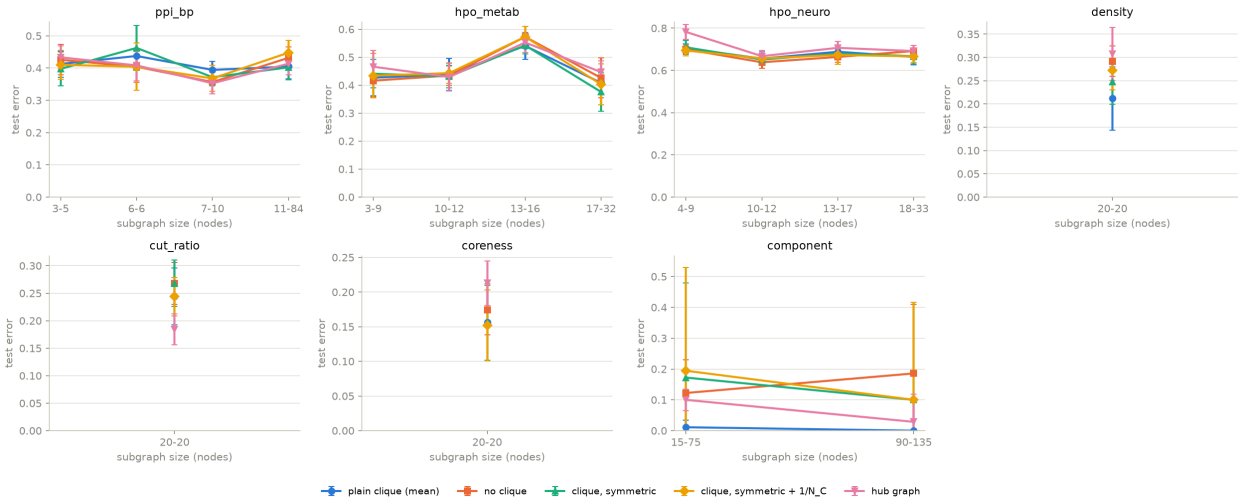


Figure 3: Test error by subgraph size, one line per case. Error is the share of test subgraphs in the group that got a wrong label, over all 10 seeds; bars are the std across seeds. On hpo_neuro, where a subgraph can have several labels, "wrong" means the predicted label set is not exactly the true one, so its error is much higher than 1−F1. density, cut_ratio and coreness have only one point because all their subgraphs have 20 nodes.

Next steps

Where we fall behind. We are close to GPEN on coreness and component, and far behind on hpo_metab, hpo_neuro, density and cut_ratio. These four are also the datasets where subgraphs share nodes: the share of subgraph pairs with at least one node in common is 0% on coreness and component and 2% on ppi_bp, against 8% on cut_ratio and 41–44% on hpo_metab, hpo_neuro and density (measured on the local data). This is only a match between two lists; the GLASS paper below suggests it is not the main cause.

What the GLASS paper shows. GLASS marks each node as inside or outside the subgraph. To save time it does not do this for one subgraph at a time: it marks a whole batch at once, and a node counts as inside if it is in any subgraph of the batch. So in GLASS too, a node in two subgraphs gets one vector per batch. The authors report that this overlap has little effect. They also report a plain GNN with no marks at all (Table 2).

	ppi_bp	hpo_metab	hpo_neuro	density	cut_ratio	coreness	component
plain GNN (no marks)	61.3	59.7	66.8	46.2	88.7	53.6	99.8
GLASS	61.9	61.4	68.5	93.0	93.5	84.0	100.0
our best	60.6	55.3	63.2	78.8	81.6	84.8	99.2

Table 2: Test micro-F1 (×100). The first two rows are from Tables 5 and 6 of the GLASS paper (10 seeds); "our best" is the best of our six models in Table 1.

On hpo, even a plain GNN beats us. Without any marks, GLASS's plain GNN gets 59.7 on hpo_metab and 66.8 on hpo_neuro, 4.4 and 3.6 points above our best. So the gap there is not about marking nodes or about overlap. One clear difference is the input: GLASS starts from node vectors pretrained on link

prediction, while our best runs learn their own vectors, which overfit (train F1 near 1.0). But in our search the pretrained SubGNN vectors did much worse than learned ones (best test 46.4 on hpo_metab and 49.9 on hpo_neuro, from about 20 trials each). So the cause on hpo is still open.

On density, the marks matter. The plain GNN gets 46.2 and GLASS 93.0. Density is the number of graph edges inside the subgraph out of all possible pairs, so the model has to know which edges stay inside. Our structural input gives each node its number of neighbors that are in any of its subgraphs, which is wrong for a node in two subgraphs, and RGCN takes a mean over neighbors, so it cannot count edges by itself.

On cut_ratio, we are below the plain GNN (81.6 against 88.7), so something more basic is missing there.

What we will try.

- **Counts on the hub.** Each hub belongs to exactly one subgraph, so it can hold that subgraph's own counts: its number of nodes, of graph edges inside it, and of graph edges leaving it. These go in as input features of the hub. On density and cut_ratio these three numbers almost give the label away, so a gain there would not show much.
- **Rebuild GLASS's plain GNN in our code.** Our backbone with no subgraph edges and pretrained input should reach about 60 on hpo_metab. If it does not, the gap is in our training or input, not in the model.
- **A check on overlap.** Plot test error against how many other subgraphs a test subgraph shares nodes with, from the saved predictions. If error does not rise, overlap can be dropped as a cause.